

Using the Unige clusters

Basic principles in brief, plus tips and tricks

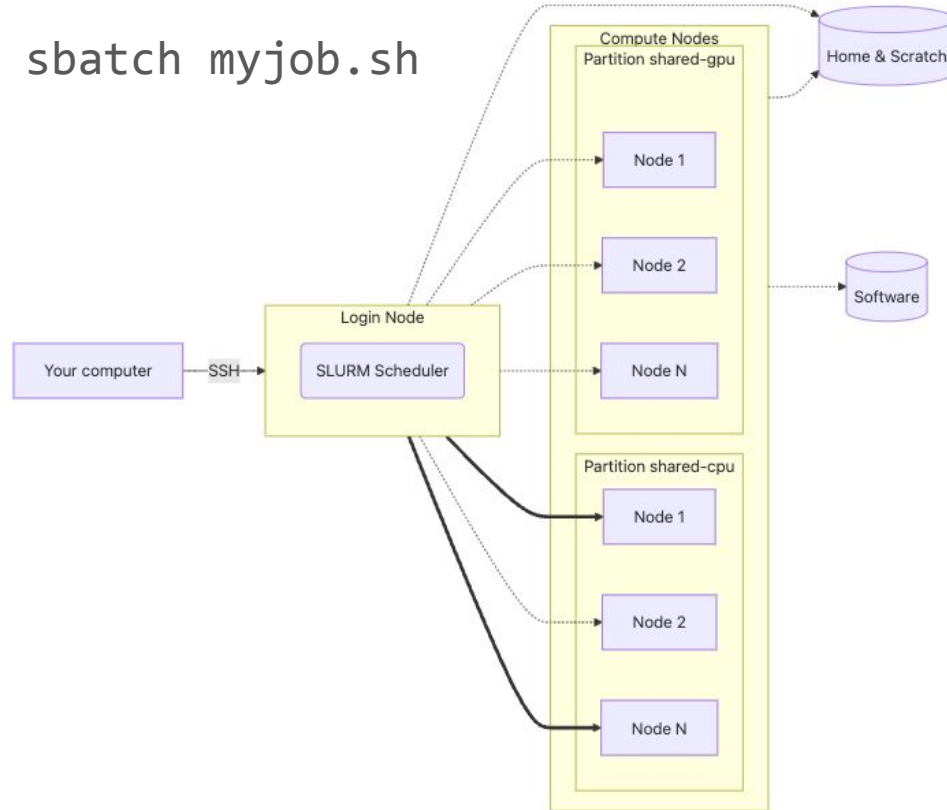
What are the clusters and how do they work?

- A cluster is many individual computers managed by a single system
- Different software is used to manage: Unige uses SLURM
- The cluster is *heterogeneous*: Individual nodes differ in hardware
- *Partitions* help manage what kind of hardware and resources you need



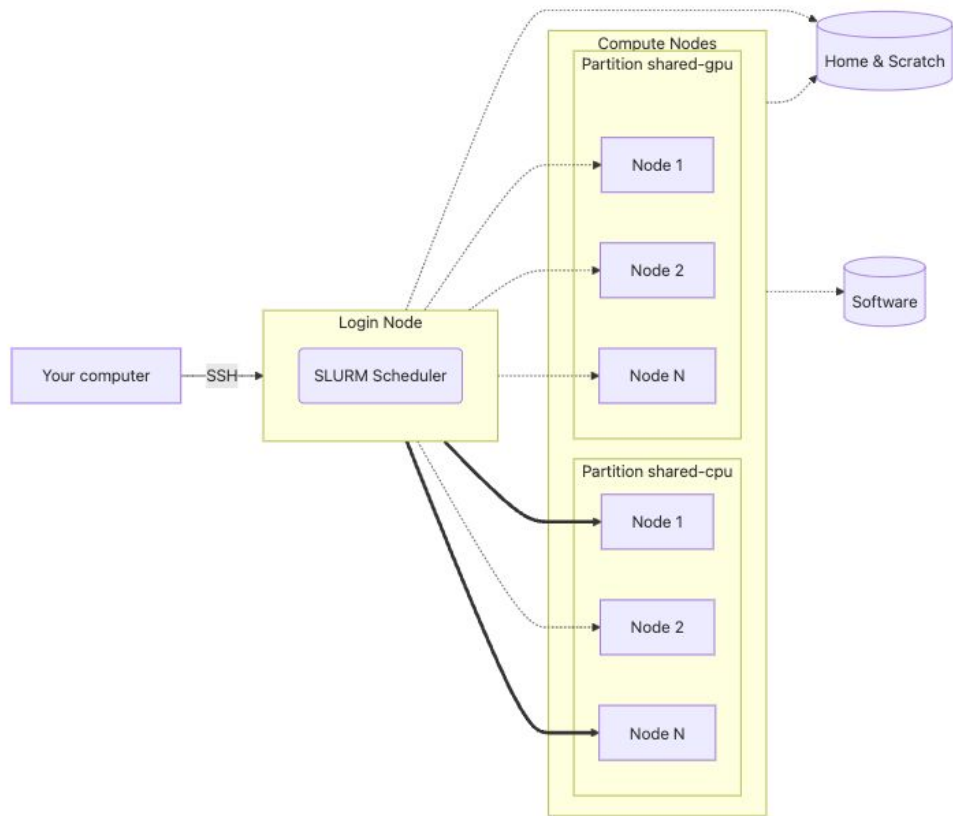
What happens when I submit a job?

`sbatch myjob.sh`



Caveats to storage on the cluster

- The BeeGFS file system used for home and scratch is temperamental
 - Accessing one large file is very fast (>1GB/s)
 - Accessing many small files is extremely slow (several minutes to load a complex python environment)
- Home is backed up, scratch is not
 - 200GB quota on Home
 - Files older than 1 month deleted on scratch
 - Use NASac for data!
- `/tmp/` offers truly temporary storage
 - Exists only **on the node you're using**, not shared
 - Sometimes sticks around on the login node, is wiped on the compute nodes after jobs



Quickly accessing the clusters

- SSH config files allow for giving nicknames to frequent connections
 - Defining a “JumpHost” tells SSH to first connect to the jump host, then connect from there to the target. Allows for bypassing the Unige firewall (see github example SSH config)
- SSH keys allow for passwordless login
 - “ssh-keygen” will create a public/private key pair
 - “ssh-add” will add the key to your keychain, and all following ssh attempts will use that key
 - “ssh-copy-id” will copy your public key to the target computer so you can log in remotely (don’t do this for the clusters)
 - See link in example SSH config to add your public key to the clusters
 - Your key can also be used to upload to github!

How are jobs submitted?

- `sbatch` uses a “job script” to submit work
 - The job script specified what resources are needed and what commands should be run
 - “Fire and forget” method: even if the resources aren’t ready now, the cluster knows what to do
- `srun` will run a single command in a job
 - Like `sbatch` but the command is passed at the command line, not in a script
 - Not particularly useful for most of our work
- `salloc` will request an allocation and connect your terminal
 - Allows for interactive work with custom resources
 - If the resources are not available, it will hang and wait until they are
- All commands share common options such as `--partition`, `--time`, `--cpus-per-task`, etc. See SLURM documentation for details.

Option 1 for cluster software: module

- The `module` command allows for loading predefined software to use
 - The cluster administrators upload these binaries for easy use
 - You can search using the command for software. Some very useful tools like FreeSurfer are available.
 - You **must load the software in your job script** before using it
 - Build-your-own Python installations are not supported
 - You can request software from the admins to be installed
- The `module` system allows for software to be installed for everyone, and can offer software that cannot be installed without root permission
- If you need something specific, sometimes this is not a good option

Option 2: Virtual environments

- Thousands of Python packages exist and the cluster cannot offer every combination of every single one
- “Virtual environments” create an isolated Python system using its own set of packages which you can modify
 - The original method is to use `venv PATH/T0/envname` to make a new environment. The environment can be activated with `source PATH/T0/envname/bin/activate`. `pip` can then install packages
 - Astral’s UV provides a drop-in replacement for the above. It’s much faster and only needs you to put “uv” before the commands
 - Anaconda (or just conda) is a more full-featured environment manager that can handle both Python and several external dependencies (e.g. CUDA). It is slow and large, but useful

Caveats to custom environments on the cluster

- The extremely slow file system makes python environments a nightmare
 - Python environments consist of thousands of tiny text files
 - Loading an environment with many packages can take several minutes
- One solution I use: compression and /tmp
 - Create an environment with `uv venv /tmp/my_env_name`
 - Activate and install packages with `uv pip install package1 package2 package3 ...`
 - Compress: `7z a -mmt64 -mx1 /home/username/my_env_name.zip /tmp/my_env_name`
 - Extract on nodes: `7z x /home/username/my_env_name.zip -o /tmp/`
 - Activate as usual, fast to unzip and fast to activate (on local storage for node)
 - Repeat the compress step if you install new packages
 - Check lab github for bashrc config that sets a “shortcut” command to do the above.

Option 3: “Containerization”

- Docker is a tool that allows you to run isolated virtual operating systems from within your computer.
 - E.g. you can run a tiny (<100MB) copy of Linux which has only the necessities for analysis of data
 - Promotes reproducible science as the “images” that you run can be shared publicly, and ensure version or OS problems don’t break code
 - Containers run these images
- Docker containers can be run on the cluster too using a tool called Apptainer
- Using containers has a high initial learning cost, but makes many analyses much easier to share and reproduce
- Similar to the compression trick, can be used to load python faster

Dealing with complex analyses

- Many analysis pipelines can take >12h to run, so they cannot be submitted in one job to the shared-cpu partition
 - Some tools that cannot use multiple CPUs are suited for the “public-longrun-cpu” partition (max. 2 CPU cores, 14 days)
 - “public-cpu” allows for run times up to 4 days with no core limit, but has few nodes and often long queues
- SLURM supports job dependencies which condition jobs on other jobs successfully running. Multiple scripts can be run in sequence like this
 - My pipeline runs data preprocessing, FreeSurfer reconstruction, and source space solving as separate steps

Knowing when to parallelize your code (Your computer)

- Parallelizing code is not easy!
- Knowing when your code can benefit from parallelization is key
- Here are two example tasks:
 - Stochastic gradient descent: Compute the gradient where you are (which direction will improve my algorithm the most?) and then run your algorithm after taking a step towards the best direction
 - Computing the Nth digit of Pi: The Nth digit of pi can be computed independently of all other digits, you want to compute 1 million digits
- Which ones can benefit from being parallelized?

Knowing when to parallelize your code (Your computer)

- When many independent tasks can be done at once, we call this “embarrassing” parallelism
- When dependencies between tasks exist, we cannot parallelize easily
- A complication: Many libraries in Python already use parallelism under the hood!
 - Numpy will use many CPU cores for some operations
 - MNE can use multiple cores if you tell it to with the “n_jobs” parameter
 - SciKit-Learn also uses multiple cores where possible (“n_jobs” applies in some cases)
 - Knowing when to parallelize requires you to watch your CPU usage while your code runs
- Modules for parallelization: multiprocessing (built-in) and joblib

Knowing when to parallelize your code (The cluster)

- If your code takes longer than a few hours to run the cluster is a good candidate
- Sample principals as local parallelization apply: Embarrassing parallelism is ideal, interdependent parallelism is very hard (see lab wiki for latter)
- Ideal cases:
 - Processing subject data in parallel, one job per subject
 - Testing many parameters for your analysis in parallel
 - Testing several different algorithms
- Your parallelism is now *across computers* instead of *across CPUs*. Every job has its own hardware
 - This means that each job can (and should!) have multiple CPU cores, allowing even already-parallel tools like Numpy and MNE to speed up still
 - The “cpus-per-task” SLURM argument is necessary!
 - Setting up / tearing down takes time: If a single task is very short (<5m) you should have jobs do multiple tasks

Exercise 1: Writing a python script to compute digits of Pi

- The lab wiki github has a function to compute the nth digit of pi (install mpmath package first)
- Write a script that will:
 - Take as a command line argument a starting and stopping digit of pi
 - Print each digit successively
 - Use argparse (best) or sys.argv to take command line input when the script is run
- Run the script on the 100th to 105th digit of Pi

Exercise 2: Running your jobs on the cluster

- Modify your script to append the digits it computes to a CSV file
- Modify your script to take a single integer from the arguments and compute the $(10,000n \text{ to } 10,000n + 9,999)$ th digits of pi
- Use the script as a basis for a SLURM job script
- Your job script should run on the “debug-cpu” partition with 1 core and a 5 minute limit
- Run a SLURM job array with 10 jobs
 - You can use the `#SBATCH --array=0-9`
 - The env variable `$SLURM_ARRAY_TASK_ID` should be used to replace the integer argument to your script